

Node Js Notes

1. Introduction to Node.js

☐ What is Node.js?

Node.js is a **runtime environment** that allows you to execute JavaScript **outside the browser** (on your computer/server).

- Normally, JavaScript runs in browsers like Chrome
- Node.js lets you run JS on the **server-side**

☐ This means you can build:

- Backend servers
 - APIs
 - Full-stack applications (with frontend + backend)
-

☐ Built on Chrome V8 Engine

Node.js uses the powerful **Chrome V8 JavaScript Engine**, which:

- Compiles JavaScript into **machine code**
 - Makes execution **very fast**
 - Is the same engine used in Google Chrome
-

☐ Where is Node.js Used?

1. Backend Development

- Create server-side logic
- Handle requests & responses
- Connect to databases

☐ Example:

- Login system
 - User authentication
 - Data processing
-

2. APIs (Application Programming Interfaces)

- Build REST APIs
- Connect frontend with backend

□ Example:

- /users → fetch users
 - /login → authenticate user
-

3. Real-Time Applications

Node.js is excellent for real-time apps because of its event-driven nature.

□ Examples:

- Chat apps (WhatsApp, Messenger)
 - Live streaming apps
 - Online gaming systems
-

□ **Key Features of Node.js**

1. Asynchronous & Non-Blocking

- Tasks do **not wait** for previous tasks to finish
- Improves performance and speed

□ Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Async Task");
}, 2000);

console.log("End");
```

□ Output:

```
Start
End
Async Task
```

- ✓ Execution does NOT stop for 2 seconds
 - ✓ Other code continues running
-

2. Event-Driven Architecture

- Node.js works on **events**
- Uses **Event Loop**

□ Example:

- User clicks button → event triggered
- Server responds → event handled

- ✓ Efficient for real-time apps
-

3. Single-Threaded (But Handles Concurrency)

- Uses **one main thread**
- But handles **multiple requests simultaneously**

□ How?

- Uses **event loop + callbacks**
- Offloads heavy tasks internally

- ✓ Lightweight & scalable
- ✓ No thread management complexity

2. How Node.js Works

□ **Event Loop Concept**

At the core of Node.js is the **Event Loop**, which allows it to handle **multiple requests efficiently using a single thread**.

□ Instead of waiting for tasks to complete, Node.js:

- Delegates heavy work
- Continues executing other code

- Comes back later when the task is done
-

☐ **Key Idea: Non-Blocking Execution**

Non-blocking = No waiting

- Traditional systems → wait for task → then continue
- Node.js → start task → move ahead → handle result later

✓ This makes Node.js:

- Fast ☐
 - Scalable ☐
 - Efficient for real-time apps
-

☐ **Step-by-Step Flow (Event Loop Working)**

1. Request Comes In

- A user sends a request (API call, file read, DB query)
-

2. Check Type of Task

Non-Blocking Task

- Simple operations (console, calculations)
- Executed immediately in main thread

Blocking / Heavy Task

- File system
- Database queries
- Network calls

☐ These are sent to:

➡ ☐ **Worker Threads (Thread Pool)**

3. Continue Execution

- Node.js **does NOT wait**
 - It continues executing next lines of code
-

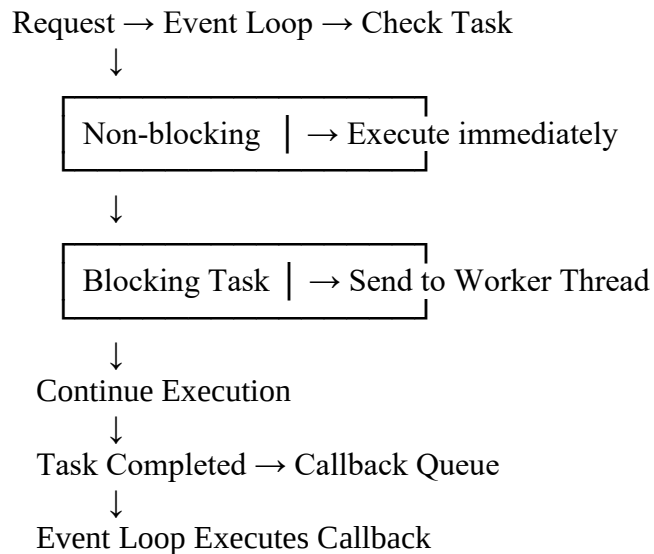
4. Task Completion

- Worker thread finishes the task
 - Sends result back to **Event Queue**
-

5. Callback Execution

- Event Loop picks completed task
 - Executes its **callback function**
-

□ Visual Flow (Simplified)



□ Practical Example

```
console.log("Start");

setTimeout(() => {
  console.log("Task Done");
```

Prathamesh Arvind Jadhav

```
}, 2000);
```

```
console.log("End");
```

☐ **Execution Flow:**

1. Start → printed
2. setTimeout → sent to Web API / worker
3. End → printed immediately
4. After 2 sec → callback executed

☐ **Output:**

Start

End

Task Done

☐ **Important Concepts**

☐ **Event Loop**

- Continuously checks:
 - Call Stack
 - Callback Queue
-

☐ **Call Stack**

- Executes synchronous code
-

☐ **Callback Queue**

- Stores completed async tasks
-

☐ **Worker Threads (Thread Pool)**

- Handles heavy operations in background
-

☐ Why This Matters

Because of Event Loop:

- Node.js handles **thousands of requests** simultaneously
- No need for multiple threads
- Perfect for:
 - APIs
 - Chat apps
 - Streaming services

3. REPL (Read–Eval–Print Loop)

☐ What is REPL?

REPL stands for:

- **Read** → takes user input
- **Eval** → executes the code
- **Print** → shows output
- **Loop** → repeats continuously

- ☐ It is an **interactive shell** provided by Node.js to quickly run JavaScript code.
-

► How to Start REPL

Open terminal and type:

node

- ☐ You will see something like:

>

Now you can write JavaScript directly.

☐ Example Usage

> 2 + 3

5

> const name = "Prathamesh";

Prathamesh Arvind Jadhav

undefined

> name

'Prathamesh'

☐ Uses of REPL

- ✓ Testing small code snippets
 - ✓ Debugging logic quickly
 - ✓ Learning JavaScript interactively
 - ✓ Checking outputs instantly
-

☐ Special Commands in REPL

Command	Description
.help	Show all commands
.exit	Exit REPL
.clear	Clear screen
.save file.js	Save session

4. Modules in Node.js

☐ What are Modules?

Modules are **reusable pieces of code**.

☐ Instead of writing everything in one file:

- Split code into multiple files
 - Export & reuse where needed
- ✓ Makes code:
- Clean
 - Maintainable
 - Scalable
-

☐ Types of Modules

1. Core Modules (Built-in)

These come **pre-installed** with Node.js.

□ Examples:

- fs → File System
- http → Create server
- path → File paths

```
const fs = require('fs');
```

2. Local Modules

- Created by **you**
- Stored in your project

```
// math.js
function add(a, b) {
  return a + b;
}
module.exports = add;
```

3. Third-Party Modules

- Installed using **npm**
- Used for advanced features

```
npm install lodash
const _ = require('lodash');
```

Export & Import (Very Important)

□ **Export (Send data from file)**

```
// file.js
function add(a, b) {
  return a + b;
}
```

```
module.exports = add;
```

☐ **Import (Use in another file)**

```
// app.js
const add = require('./file');

console.log(add(2, 3)); // 5
```

☐ **Export Multiple Values**

```
// math.js
function add(a, b) { return a + b; }
function sub(a, b) { return a - b; }

module.exports = { add, sub };
const math = require('./math');

console.log(math.add(5, 2));
console.log(math.sub(5, 2));
```

☐ **Key Points**

- `module.exports` → used to export
- `require()` → used to import
- Each file in Node.js = **separate module**
- Modules help in **code reusability**

5. Core Modules (Important) in Node.js

Core modules are **built-in modules** — no installation required ☐

1. fs (File System)

☐ Used to **read, write, update, delete files**

```
const fs = require('fs');

// write
fs.writeFileSync('file.txt', 'Hello');

// read
const data = fs.readFileSync('file.txt', 'utf-8');
console.log(data);
```

□ **Key Points:**

- `writeFileSync()` → writes file (blocking)
 - `readFileSync()` → reads file (blocking)
 - Works with both **Sync & Async methods**
-

2. path Module

□ Helps to **handle file & directory paths**

```
const path = require('path');
```

```
console.log(path.basename(__filename)); // file name  
console.log(path.extname(__filename)); // extension
```

□ **Useful Methods:**

- `basename()` → file name
 - `extname()` → extension
 - `join()` → join paths
 - `resolve()` → absolute path
-

3. os Module

□ Gives **system-level information**

```
const os = require('os');
```

```
console.log(os.platform()); // OS platform  
console.log(os.cpus());    // CPU info
```

□ **Use Cases:**

- Detect OS
 - Performance monitoring
 - System diagnostics
-

4. http Module (Very Important)

□ Used to create **web servers without frameworks like Express.js**

Prathamesh Arvind Jadhav

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.write('Hello World');
  res.end();
});

server.listen(3000);
```

► How it Works:

1. Client sends request
2. Server receives (req)
3. Server sends response (res)

❑ Open browser:

<http://localhost:3000>

5. events Module

❑ Used for **event-driven programming**

```
const EventEmitter = require('events');

const emitter = new EventEmitter();

emitter.on('msg', () => {
  console.log('Event triggered');
});

emitter.emit('msg');
```

❑ **Concept:**

- on() → listen to event
- emit() → trigger event

✓ Backbone of Node.js architecture

6. File System (Deep Understanding)

☐ Sync vs Async

Synchronous (Blocking)

- Executes line by line
- Stops execution until task completes

```
const fs = require('fs');
const data = fs.readFileSync('file.txt', 'utf-8');
console.log(data);
```

Asynchronous (Non-Blocking) ☐ (Preferred)

- Does NOT block execution
- Uses callback

```
const fs = require('fs');

fs.readFile('file.txt', 'utf-8', (err, data) => {
  console.log(data);
});
```

☐ Difference

Feature	Sync	Async
Execution	Blocking	Non-blocking
Performance	Slow	Fast
Use Case	Small tasks	Real apps

- ☐ Always prefer **Async in production**
-

7. Streams & Buffers

☐ Buffers

☐ Used to store **binary data (raw memory)**

- Needed when working with:
 - Files
 - Network data
 - Streams

```
const buffer = Buffer.from('Hello');  
console.log(buffer);
```

☐ Streams

- ☐ Process data **in chunks (piece by piece)**
- ☐ Instead of loading full file in memory

- ✓ Memory efficient
 - ✓ Faster for large data
-

☐ Types of Streams

Type	Description
Readable	Read data
Writable	Write data
Duplex	Read + Write
Transform	Modify data

☐ Example: Read Stream

```
const fs = require('fs');  
  
const readStream = fs.createReadStream('file.txt');  
  
readStream.on('data', chunk => {
```

```
    console.log(chunk);  
  });
```

❑ Stream Flow

File → Stream → Chunk → Process → Output

❑ Real-Life Example

- Watching YouTube ❑
 - ❑ Video loads in chunks (streaming)
- File download ❑
 - ❑ Data comes in parts, not all at once

8. Callbacks in Node.js

❑ What is a Callback?

A **callback** is a function passed as an argument to another function and executed later.

```
function greet(name, callback) {  
  console.log(name);  
  callback();  
}
```

```
function sayBye() {  
  console.log("Bye!");  
}
```

```
greet("Prathamesh", sayBye);
```

❑ Output:

```
Prathamesh  
Bye!
```

❑ Key Points:

- Enables **asynchronous programming**
 - Used heavily in Node.js (especially older code)
 - Can lead to ❑ *callback hell* if overused
-

9. Promises

□ What is a Promise?

A **Promise** represents a value that may be:

- Available now
- Available later
- Or never

```
const promise = new Promise((resolve, reject) => {  
  resolve("Success");  
});
```

□ Using Promise

```
promise  
  .then(result => console.log(result))  
  .catch(err => console.log(err));
```

□ Output:

Success

□ States of Promise:

- **Pending** → initial
 - **Resolved (Fulfilled)** → success
 - **Rejected** → failure
-

10. Async / Await

□ What is Async/Await?

A modern way to handle **Promises** that looks like synchronous code.

```
async function getData() {  
  const data = await promise;  
  console.log(data);  
}
```

```
getData();
```

❑ **Output:**

Success

❑ **Key Points:**

- `async` → makes function return a promise
 - `await` → waits for promise to resolve
 - Cleaner than `.then()` chaining
-

❑ **Callback vs Promise vs Async/Await**

Feature	Callback	Promise	Async/Await
Readability	Low	Medium	High
Error Handling	Difficult	Better	Best
Usage	Old	Modern	Latest

11. Error Handling

❑ **Try-Catch**

```
try {  
  throw new Error("Error");  
} catch (err) {  
  console.log(err.message);  
}
```

❑ **Output:**

Error

❑ **Async Error Handling**

```
async function test() {  
  try {  
    await Promise.reject("Failed");  
  }  
}
```

```
    } catch (err) {  
      console.log(err);  
    }  
  }  
}  
  
test();
```

☐ **Key Points:**

- Use try-catch with async/await
 - Use .catch() with promises
 - Always handle errors in production ☐
-

12. Process Object

☐ process is a **global object** that provides information about the current Node.js process.

☐ **Examples**

```
console.log(process.argv);  
console.log(process.env);
```

☐ **Uses:**

- process.argv → command-line arguments
- process.env → environment variables

☐ **Example:**

```
node app.js hello  
console.log(process.argv[2]); // hello
```

13. Global Objects

☐ These are available everywhere (no need to import)

☐ **Important Globals**

1. `__dirname`

- ☐ Current directory path

```
console.log(__dirname);
```

2. `__filename`

- ☐ Current file path

```
console.log(__filename);
```

3. `global`

- ☐ Global namespace object

```
global.myVar = "Hello";  
console.log(myVar);
```

4. `setTimeout()`

- ☐ Executes code after delay

```
setTimeout(() => {  
  console.log("Hello after 2 sec");  
}, 2000);
```

☐ **Final Understanding**

Async Handling Flow:

- ☐ Callback → Promise → Async/Await (evolution)

Error Handling:

- ☐ Always handle errors properly

Process & Globals:

- ☐ Help interact with system & environment

14. package.json in Node.js

☐ What is package.json?

It is the **heart of any Node.js project** ❤️ ☐

☐ Created using:

npm init -y

☐ What it Stores:

- ☐ Project metadata (name, version)
 - ☐ Dependencies (installed packages)
 - ☐ Scripts (custom commands)
 - ☐ Author info
-

☐ Example

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "lodash": "^4.17.21"
  }
}
```

15. NPM (Node Package Manager)

☐ What is npm?

- Default package manager for Node.js
 - Used to install & manage libraries
-

☐ Install Package

Prathamesh Arvind Jadhav

npm install package-name

☐ Example:

npm install express

☐ **Key Features:**

- Install dependencies
 - Update packages
 - Manage versions
 - Run scripts
-

16. CLI (Command Line App)

☐ **What is CLI?**

A program that runs via **terminal/command line**

☐ **Example**

```
const args = process.argv;  
console.log(args);
```

► **Run:**

node app.js hello world

☐ Output:

['node', 'app.js', 'hello', 'world']

☐ **Use Cases:**

- Build tools (like npm, git)
 - Automation scripts
 - Custom utilities
-

17. Environment Variables

☐ What are they?

Variables stored **outside code** for configuration

PORT=3000

☐ Access in Node.js

```
console.log(process.env.PORT);
```

☐ Why Use?

- ✓ Security (hide secrets)
 - ✓ Flexibility (change config without code)
 - ✓ Environment-based setup (dev/prod)
-

18. Threading Model

☐ Nature of Node.js

- Single-threaded main execution
 - Uses **event loop** internally
 - Built on **libuv** (handles async tasks)
-

☐ How it Handles Work:

☐ Main Thread:

- Handles event loop
- Executes callbacks

☐ Worker Threads:

- Handle heavy tasks (file, DB, network)
-

☐ Key Concept:

- ✓ Single thread $\neq$ single task
 - ✓ Handles **multiple operations concurrently**
-

19. Event Loop Phases (Advanced)

The Event Loop runs in **phases** ☐

☐ **Phases:**

1. Timers

- Executes `setTimeout()` / `setInterval()`
-

2. I/O Callbacks

- Handles completed I/O operations
-

3. Idle / Prepare

- Internal use (Node.js)
-

4. Poll

- Waits for new I/O events
 - Executes I/O callbacks
-

5. Check

- Executes `setImmediate()`
-

6. Close Callbacks

- Handles closing events (e.g., socket close)

☐ **Flow**

Timers → I/O → Idle → Poll → Check → Close → Repeat

☐ **20. Debugging in Node.js**

☐ **Start Debug Mode**

`node inspect app.js`

☐ **Other Methods:**

- `console.log()` (basic debugging)
 - Use Chrome DevTools
 - VS Code debugger
-

☐ **Tips:**

- ✓ Breakpoints help track bugs
 - ✓ Inspect variables step-by-step
-

21. Best Practices

1. Use Async Over Sync

- ✓ Avoid blocking code
 - ✓ Improves performance
-

2. Handle Errors Properly

- ✓ Use try-catch
- ✓ Always add `.catch()`

3. Write Modular Code

- ✓ Split into multiple files
 - ✓ Use reusable modules
-

4. Use Environment Variables

- ✓ Store configs securely
 - ✓ Avoid hardcoding
-

5. Avoid Blocking Operations

- ✓ Use streams for large data
- ✓ Use async APIs